Implementing and Evaluating Word Count Service

1st Jules Anseaume
Master in Embedded Systems
TU Eindhoven
Eindhoven, Netherlands
j.w.i.l.anseaume@student.tue.nl

2st Angelo Filiol de Raimond
Master in Computer Science and Engineering
TU Eindhoven
Eindhoven, Netherlands
angelo.filiolderaimond@gmail.com

***Abstract*—This report describes the implementation and evaluation of a Word Count Service. It covers functional and non-functional requirements, system architecture, choice of architectural styles, experimental setup, and results obtained.**

***Index Terms*—Word Count Service, Architecture, Requirements, P2P, Layered, Evaluation**

I. Phase 1: Requirements and Architecture

A. Requirements and stakeholders

1) Functional Requirements (FR):

- Allow the user to query how many times a word appears in a text stored on a server.
- Cache the results in an in-memory database to improve response time.

2) Non-Functional Requirements (NFR):

- **Latency:** The system should provide results with minimal delay.
- **Scalability:** The system should support multiple simultaneous client requests without performance degradation.

3) Stakeholders:

- **Users:** Request the occurrence of a word in a text.
- **Engineers / System Administrators:** Implement and maintain the service.

B. Architecture

This diagram illustrates a three-tier Client-Server architecture for a word-counting service with caching.

Main Components: **1. Client**

- Entry point of the system where users send their requests
- Initiates word-count requests by specifying a keyword and a reference to the text file
- Receives responses containing the number of occurrences of the searched word

2. Server (Application Server)

- Central component that orchestrates request processing
- Handles word-count requests from clients
- Queries Redis to check if the result is already cached
- If not found in the cache, performs the count on the text file and stores the result in Redis
- Returns the response to the client

3. Redis (In-Memory Database)

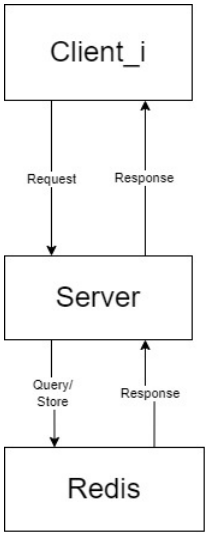


Fig. 1: System architecture of simple Client server Architecture

- Caching system that stores results of previous queries
- Reduces execution latency for identical requests
- Can also be used to analyze "hot keywords" (most frequently searched keywords)

Connectors and Communication Flows: **Client ↔ Server**

- **Request**: The client sends a request containing the keyword to search and the file reference
- **Response**: The server returns the number of occurrences found

Server ↔ Redis

- **Query/Store**: The server queries Redis to retrieve a cached result or stores a new result after processing
- **Response**: Redis returns either the cached result or an indication that the result does not exist

Advantages of this Architecture:

- **Scalability**: Clear separation of responsibilities allows each component to scale independently
- **Performance**: Caching significantly reduces response time for repeated requests
- **Fault Tolerance**: Modular architecture facilitates failure management

Prepared within the MSc Computer Science and Engineering program, TU Eindhoven, 2025.

C. Architectural Style Trade-off Analysis

Peer-to-Peer (P2P):

- **How:** Each client (peer) processes texts locally and shares results with other peers. The cache is distributed among clients.
- **Advantage:** Fault-tolerant; no single point of failure.
- **Disadvantage:** Results may be unavailable if a peer disconnects; cache synchronization adds complexity.

Layered:

- **How:** Client interface layer handles requests, logic layer implements counting, storage/cache layer stores results in Redis. Layers communicate only with adjacent layers.
- **Advantage:** Modular, easier testing, maintenance, and future extension.
- **Disadvantage:** Can be unnecessarily complex for a small-scale project.

II. Phase 2: Implementation of the Service

A. The implementation

The Service is implemented in 2 separate parts using the rpyc library:

1) Libraries: The libraries used are rpyc, which creates a connection between the client and server to send over requests. The redis library is used as a caching system, which allows for handling requests faster by simply caching previous results, and retrieving these upon future requests if a duplicate requests has been handled.

2) Client: The Client service (which can be duplicated) creates requests to the server, first by connecting to the server, and then continuously sending requests. It does so by using the ”exposed_wordcountingservice” method, which it can connect to through rpyc.

3) Server: The Server service (which can also be duplicated, but in this case shouldn’t be) receives requests from clients, handles it by looking for a word in a given reference, then returns the count. In case the redis caching system is used, it also tries to retrieve cached results from the cache. Should the result not have been cached yet, the server computes the word count of the word in the reference.

4) Results: In 2, we can see the result of phase 2 using 3 clients. As we can see, the requests get fulfilled, and results are printed.

```
client   | INFO:__main__:Request for 'escape' in file 'Fable.txt' completed in 0.0011 seconds


client3  | INFO:__main__:3
client2  | INFO:__main__:0


client   | INFO:__main__:0
client2  | INFO:__main__:Request for 'clever' in file 'Fable.txt' completed in 0.0043 seconds
client3  | INFO:__main__:Request for 'strength' in file 'Fable.txt' completed in 0.0032 seconds




client   | INFO:__main__:Request for 'danger' in file 'Fable.txt' completed in 0.0019 seconds
client2  | INFO:__main__:6
```

Fig. 2: Result of Phase 2 implementation

B. Client requests total runtime

In this section, we specify the total runtime of two design choices; a server without a caching system, and another with one. Note that these have been checked by hand. Hence, they are not perfect.

1) Runtime of multiple clients requesting without using caching: The time it takes to perform 28 duplicate requests per server is on the order of 2.43 seconds. This can be considered a pretty decent time for a simple structure on texts which are at most 10000 words

2) Runtime of mulitple clients requesting using caching: The time it takes to perform 28 duplicate requests per server is on the order of 2.36 seconds. This is still quite a long duration for simple requests of word counting a document and sending back the result, especially with a cache that stores pre-existing requests. Thus, the time gained cannot be considered that impressive, though there is still a tiny improvement, which upon millions of requests is more efficient. Because the timings do not differ that much between using a cache and without, we will show the results per word search using the cached system in Figure 2:

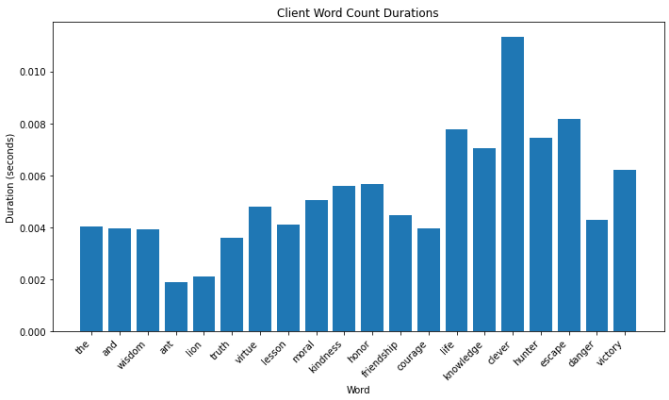


Fig. 3: Timings for each word search in the Phase 2 implementation

To analyze the results, we need to think about the results for each word. The word ”the”, being small, would be faster to find, even if in larger quantities. Words such as ”knowledge”, ”clever” or ’victory” are more complex, and thus, even with reduced numbers, still take more time. Furthermore, since each word hasn’t yet been cached, these results aren’t skewered by results being cached.

III. Phase 3: Scalability & Load Balancing

Scalability is an important element of distributed systems. A singular server not only becomes a bottleneck of requests, but also a point of failure. Hence, it is important to use multiple servers that share the load of requests. However, the requests need to be shared towards multiple servers, which for a client would require to know each and every server available. This becomes another problem, solved by the load balancer.

A. Need for load balancer

A load balancer, exactly as the name suggests, balances a load, which in this case represents requests coming from 3 separate clients. As the clients don't need to know all servers, the load balancer is the point of contact between the clients and servers, as seen in the Phase 3 system architecture diagram. In this way, the clients only need to know and connect to the load balancer, with the load balancer sending requests over the Transport Layer to the server, and back. The two load balancing algorithms chosen are Round Robin, and Two-Choice.

B. System Architecture

In Phase 3, the new system architecture looks like the figure seen below:

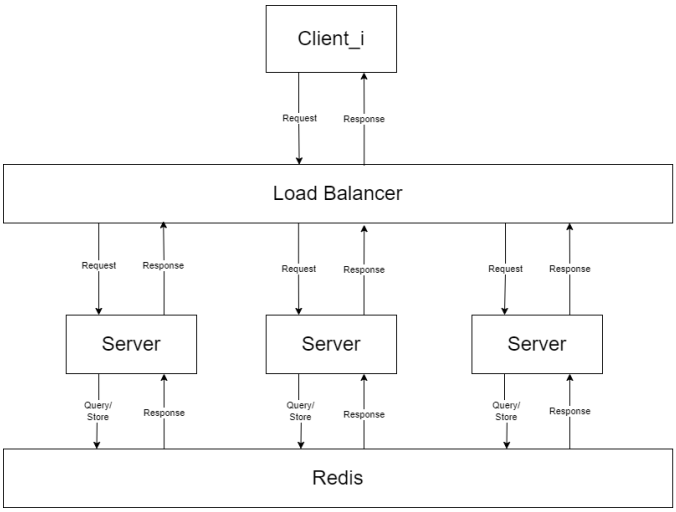


Fig. 4: Phase 3 System Architecture

Note that Client_i represents a client, with our implementation allowing for multiple clients as well as multiple servers. The Load balancer now acts as a single point of contact of the client(s), and can send the request to any of the 3 servers. All 3 servers share the same caching system, as this allows servers to share pre-computed results, creating even more efficiency.

C. First load balancing algorithm

The Round Robin algorithm uses a list of servers and iterates over them one by one to transfer requests. This is one of the simpler algorithms, with however very efficient handling of requests, as all servers have a theoretically balanced load.

D. Second load balancing algorithm

The Two-Choice algorithm uses a list of servers, samples two at random, and transfers the request to the server with the least amount of current connections. This is also a very simple algorithm, and might ignore an entire server.

E. Comparison: Round Robin

1) Proof of functionality: Here, you will see the entire setup being run using the Round Robin algorithm, with proof in Figure 3:

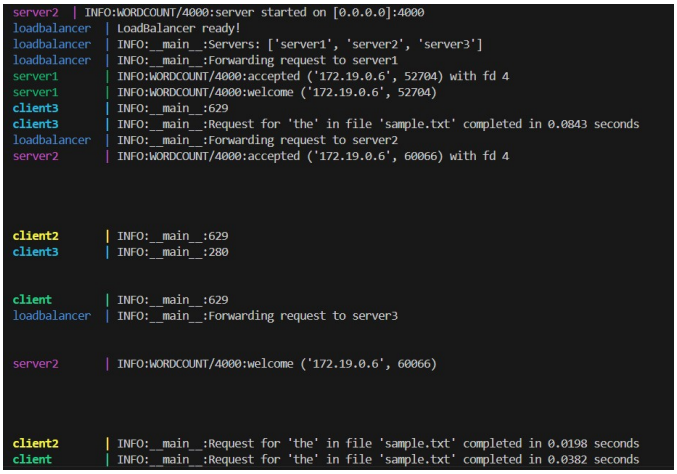


Fig. 5: Proof of Functionality using Round Robin algorithm

2) Runtime of word requests: In Figure 4, we can also see runtime of certain word requests:

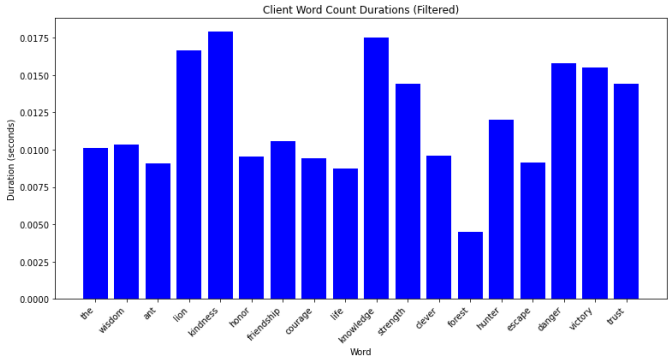


Fig. 6: Runtime of word requests using Round Robin algorithm

If we compare the runtime of these words with the runtime of Phase 2, we can see that on average it takes more time. While this sounds counterintuitive, it is logical considering that requests take more time to be transferred. Our implementation may not include enough clients to make this work, but we can see that the difference is extremely low, even with added overhead. If we consider the picking of a server, the transferring of the request to the right server and keeping the requests available for later, the difference in runtime can not only be explained, but we can consider the runtime of larger files as more efficient, if the overhead accounts for more than the time to count the number of words (which implies very large text files). For simple processes, a load balancer engages in inefficiencies. However, for larger processes (e.g. visualizations, word counting in large text files such as books, or general computing tasks), the implementation of a load balancer would indeed result in smaller delays and more requests being handled.

F. Comparison: Two-Choice

1) Proof of functionality: Here, you will see the entire setup being run using the Round Robin algorithm, with proof in

Figure 5:

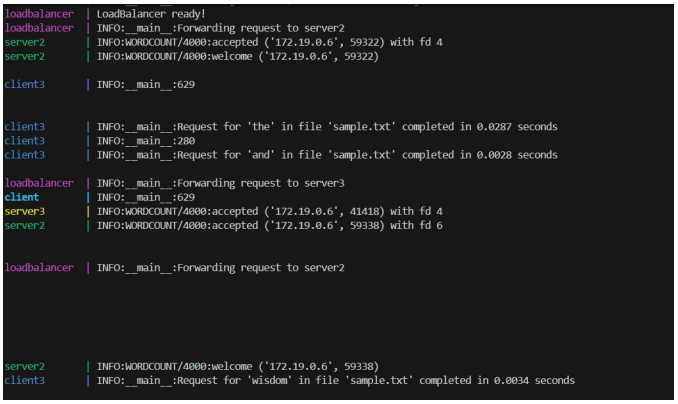


Fig. 7: Proof of Functionality using Round Robin algorithm

2) Runtime of word requests: In Figure 6, we can also see runtime of certain word requests:

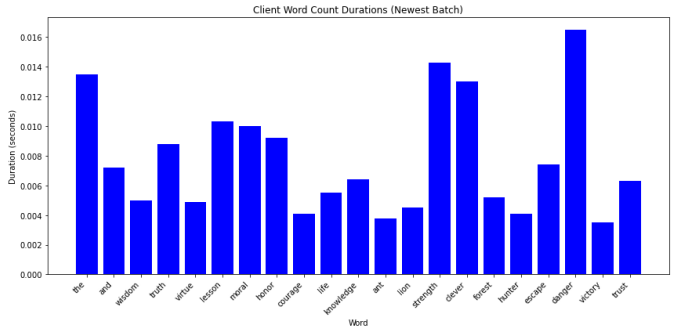


Fig. 8: Runtime of word requests using Round Robin algorithm

Just like for Round Robin, the runtime is higher, but it is clear that if we consider the overhead of all actions to decide on a server, the balancing of loads would be more efficient, should the requests be more complex and time extensive.

IV. Phase 4: Load Balancing with Fault Tolerance

A. Explanation of failure detection mechanism

Failures in servers are some of the primary problems of the internet today. This is apparent from recent AWS servers being unavailable. This wouldn't be an issue however with added Fault Tolerance. Fault Tolerance is the process of handling malfunctioning services, by applying checking methods to ensure proper functioning. In our situation, we opted for a basic approach of routine health checks for each server, and the implementation of removing servers which are considered "unhealthy", aka down or not responding to requests.

1) System Architecture in case of Failure: Let us assume that Server 2 fails, and is now unavailable. The server has thus been removed from the healthy servers, and the diagram from Phase 3 would transform into a diagram like this:

B. Failure implementation

Our implementation follows a very simple routine using separate threads for requests, and a thread for health checkups.

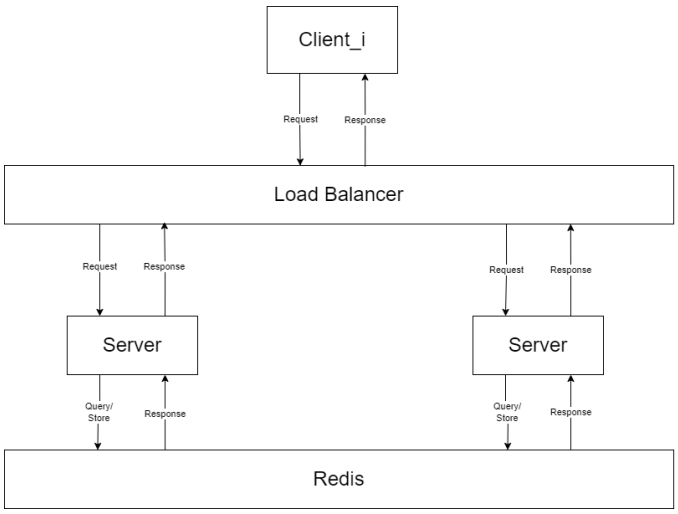


Fig. 9: Phase 4 System Architecture

1) Health checkups: Our Load Balancer is initialized with a handler, which handles requests whenever they are entered, as well as a continuously looping health check, represented by the health_check(server_name, server_port). This checks whether a connection can be established with a server through the given server name and port. Should this health check fail, we know the server is down, and remove it from the list of healthy servers. If it works, nothing occurs. This implementation ensures that even though certain requests might fail due to the time interval between health checks, on a permanent service, would improve response time drastically in case of a server failure.

2) Server death: To kill a server without creating issues with the clients, we kill the serve before any clients send requests. This way, the server doesn't accidentally cause errors to our client, resulting in early termination.

3) Proof of failure: In the figure below, you can see the server check for server 1 eventually fails due to it being killed early through the `docker compose down server1` command.

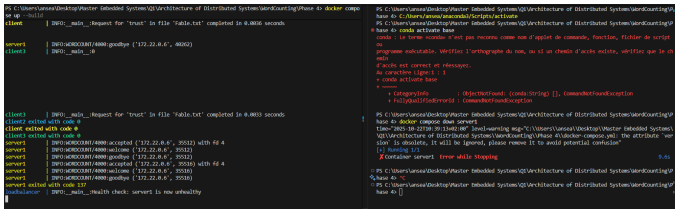


Fig. 10: Proof of server 1 being checked as unhealthy

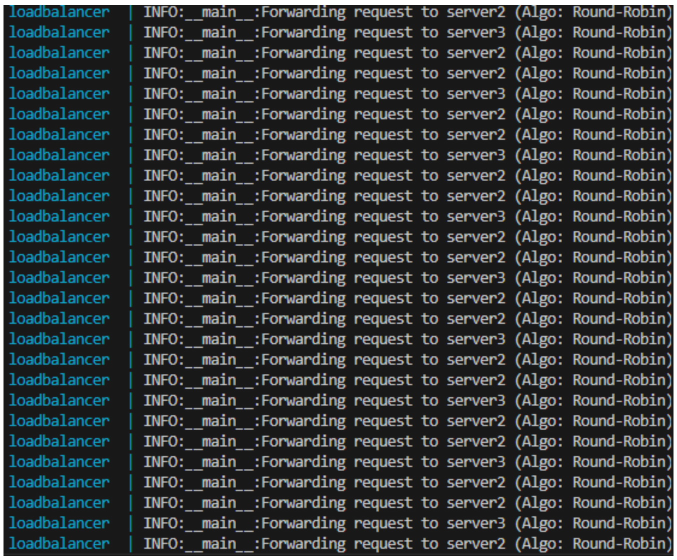
We can see the load balancer log message informing us that `server1` has been checked as unhealthy, which implies that it is removed from further requests.

C. Fault tolerance first load balancing algorithm

The fault tolerance on Round Robin uses a simple algorithm meant to iterate over all healthy servers, instead of the list

of all servers. This way, no unhealthy servers are accessed for requests. If an unhealthy server exists, it is removed from the list of healthy servers, and will be ignored. This also means that we are now hashing the request index over `len(Servers) - 1` in our case.

1) Results: In the figure below, you can see the logs of the `Load Balancer`, which clearly shows that requests are only forwarded to functioning servers (in this case servers 2 and 3).



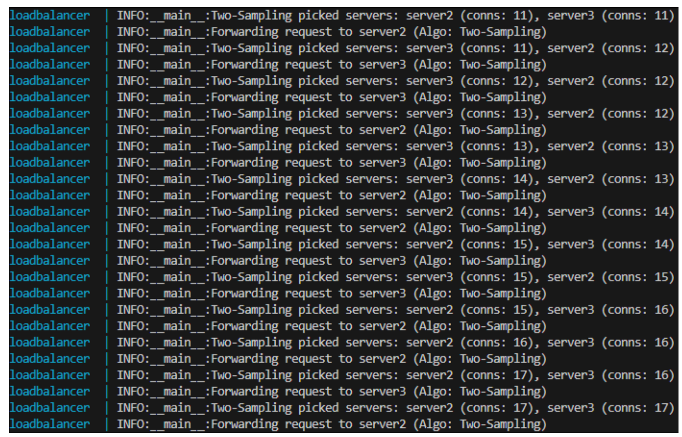
```
loadbalancer  | INFO:__main__:Forwarding request to server2 (Algo: Round-Robin)
loadbalancer  | INFO:__main__:Forwarding request to server3 (Algo: Round-Robin)
loadbalancer  | INFO:__main__:Forwarding request to server2 (Algo: Round-Robin)
loadbalancer  | INFO:__main__:Forwarding request to server2 (Algo: Round-Robin)
loadbalancer  | INFO:__main__:Forwarding request to server3 (Algo: Round-Robin)
loadbalancer  | INFO:__main__:Forwarding request to server2 (Algo: Round-Robin)
loadbalancer  | INFO:__main__:Forwarding request to server2 (Algo: Round-Robin)
loadbalancer  | INFO:__main__:Forwarding request to server3 (Algo: Round-Robin)
loadbalancer  | INFO:__main__:Forwarding request to server2 (Algo: Round-Robin)
loadbalancer  | INFO:__main__:Forwarding request to server2 (Algo: Round-Robin)
loadbalancer  | INFO:__main__:Forwarding request to server3 (Algo: Round-Robin)
loadbalancer  | INFO:__main__:Forwarding request to server2 (Algo: Round-Robin)
loadbalancer  | INFO:__main__:Forwarding request to server2 (Algo: Round-Robin)
loadbalancer  | INFO:__main__:Forwarding request to server3 (Algo: Round-Robin)
loadbalancer  | INFO:__main__:Forwarding request to server2 (Algo: Round-Robin)
loadbalancer  | INFO:__main__:Forwarding request to server2 (Algo: Round-Robin)
loadbalancer  | INFO:__main__:Forwarding request to server3 (Algo: Round-Robin)
loadbalancer  | INFO:__main__:Forwarding request to server2 (Algo: Round-Robin)
loadbalancer  | INFO:__main__:Forwarding request to server2 (Algo: Round-Robin)
loadbalancer  | INFO:__main__:Forwarding request to server3 (Algo: Round-Robin)
loadbalancer  | INFO:__main__:Forwarding request to server2 (Algo: Round-Robin)
loadbalancer  | INFO:__main__:Forwarding request to server2 (Algo: Round-Robin)
loadbalancer  | INFO:__main__:Forwarding request to server3 (Algo: Round-Robin)
loadbalancer  | INFO:__main__:Forwarding request to server2 (Algo: Round-Robin)
loadbalancer  | INFO:__main__:Forwarding request to server2 (Algo: Round-Robin)
loadbalancer  | INFO:__main__:Forwarding request to server3 (Algo: Round-Robin)
loadbalancer  | INFO:__main__:Forwarding request to server2 (Algo: Round-Robin)
```

Fig. 11: Load Balancer Fault Tolerance Proof using Round Robin

D. Fault tolerance second load balancing algorithm

The fault tolerance on Two-Choice uses a simple algorithm as well, which randomly samples 2 servers over all healthy servers, instead of the list of all servers. This way, no unhealthy servers are accessed for requests either. Just like for the Round Robin fault tolerance implementation, we thus sample over `len(Servers) - 1` in our case. We added an extra guard, which states that if there are less than 2 healthy server (implying only one active server), that server is chosen automatically.

1) Results: In the figure below, you can see the logs of the `Load Balancer`, which clearly shows that requests are only forwarded to functioning servers (in this case servers 2 and 3).



```
loadbalancer  | INFO:__main__:Two-Sampling picked servers: server2 (conns: 11), server3 (conns: 11)
loadbalancer  | INFO:__main__:Forwarding request to server2 (Algo: Two-Sampling)
loadbalancer  | INFO:__main__:Two-Sampling picked servers: server3 (conns: 11), server2 (conns: 12)
loadbalancer  | INFO:__main__:Forwarding request to server3 (Algo: Two-Sampling)
loadbalancer  | INFO:__main__:Two-Sampling picked servers: server3 (conns: 12), server2 (conns: 12)
loadbalancer  | INFO:__main__:Forwarding request to server3 (Algo: Two-Sampling)
loadbalancer  | INFO:__main__:Two-Sampling picked servers: server3 (conns: 13), server2 (conns: 12)
loadbalancer  | INFO:__main__:Forwarding request to server2 (Algo: Two-Sampling)
loadbalancer  | INFO:__main__:Two-Sampling picked servers: server3 (conns: 13), server2 (conns: 13)
loadbalancer  | INFO:__main__:Forwarding request to server3 (Algo: Two-Sampling)
loadbalancer  | INFO:__main__:Two-Sampling picked servers: server3 (conns: 14), server2 (conns: 13)
loadbalancer  | INFO:__main__:Forwarding request to server2 (Algo: Two-Sampling)
loadbalancer  | INFO:__main__:Two-Sampling picked servers: server2 (conns: 14), server3 (conns: 14)
loadbalancer  | INFO:__main__:Forwarding request to server2 (Algo: Two-Sampling)
loadbalancer  | INFO:__main__:Two-Sampling picked servers: server2 (conns: 15), server3 (conns: 14)
loadbalancer  | INFO:__main__:Forwarding request to server3 (Algo: Two-Sampling)
loadbalancer  | INFO:__main__:Two-Sampling picked servers: server3 (conns: 15), server2 (conns: 15)
loadbalancer  | INFO:__main__:Forwarding request to server3 (Algo: Two-Sampling)
loadbalancer  | INFO:__main__:Two-Sampling picked servers: server2 (conns: 15), server3 (conns: 16)
loadbalancer  | INFO:__main__:Forwarding request to server2 (Algo: Two-Sampling)
loadbalancer  | INFO:__main__:Two-Sampling picked servers: server2 (conns: 16), server3 (conns: 16)
loadbalancer  | INFO:__main__:Forwarding request to server2 (Algo: Two-Sampling)
loadbalancer  | INFO:__main__:Two-Sampling picked servers: server2 (conns: 17), server3 (conns: 16)
loadbalancer  | INFO:__main__:Forwarding request to server3 (Algo: Two-Sampling)
loadbalancer  | INFO:__main__:Two-Sampling picked servers: server2 (conns: 17), server3 (conns: 17)
loadbalancer  | INFO:__main__:Forwarding request to server2 (Algo: Two-Sampling)
```

Fig. 12: Load Balancer Fault Tolerance Proof using Round Robin